CREDNEXA

WEBSITE PROJECT REPORT

Purpose • Implementation • Architecture • Testing • Future Scope

Project type	Educational loan-estimation web application
Frontend	HTML, CSS, JavaScript
Backend	Node.js and Express API
Data storage	SQLite database
Authentication	Password hashing and token-based authentication

Documentation note: this report describes a recreated educational project package. It is not a claim that the original Netlify website source was downloaded or inspected. Live bank, lender, and AI integrations are not included unless separately implemented.

2. Executive Summary

Overview

CredNexa is a web-based educational loan-estimation application intended to help a visitor understand an approximate borrowing scenario before approaching a lender. It combines a loan calculator and account-related functions in one interface. It is a demonstration tool, not a lender’s official approval system.

Main features

The frontend presents the interface, collects calculator inputs, and displays results. An Express backend exposes API endpoints. SQLite provides local persistent storage for account and saved-estimate data. Authentication-related functionality is included in the prepared implementation.

Value

A guided estimate can make loan concepts easier to understand, encourage comparison of possible amounts and repayment periods, and help users prepare questions for a financial institution.

Project boundary

This is a recreated full-stack implementation based on the CredNexa concept. It is not the original website source repository and does not connect to banks, credit bureaus, real-time lender offers, or a live AI service.

3. Background and Problem Statement

Background

Loan information is often spread across lender pages, forms, and financial terminology. First-time applicants may find it difficult to relate principal, interest, repayment period, and monthly instalments. A simple calculator offers a low-friction first step.

Problem

A user needs a straightforward way to enter a loan amount, interest rate, and tenure, then view a repayment estimate and optionally save the scenario. Without a clear tool, users may calculate manually or misunderstand how tenure and interest affect repayment.

Proposed solution

CredNexa combines a simple user experience with server-side routes and a database. It demonstrates the complete workflow: user input, client-side interaction, API request, server processing, storage, and response.

Assumptions

The user enters plausible values and understands that actual loan terms vary by lender and applicant. The displayed result is only an estimate.

4. Why the CredNexa Website Is Important

Financial awareness

It helps users see how borrowing costs can change when principal, interest rate, or repayment period changes. An estimate can prompt users to think about affordability before applying.

Convenience

A browser-based interface can be accessed without installing a dedicated desktop program. Clear labels and immediate results reduce the effort needed for a basic calculation.

Planning support

Users can compare hypothetical scenarios, such as a smaller loan over a shorter period against a larger loan over a longer period. This supports planning but does not replace a personal budget or professional advice.

Educational value

The project demonstrates common software-engineering concepts: form handling, validation, API design, authentication, persistence, and responsive interface design.

Responsible use

Results must be labelled as estimates. The website should not imply guaranteed approval or claim to provide regulated financial advice.

5. Goals and Objectives

Primary objective

Build a demonstration loan-estimation website with a browser interface and server-side API.

Functional objectives

Provide a loan estimate workflow; support registration and login-related routes; allow authenticated users to save or delete estimates; store application data in SQLite; and return structured API responses.

Quality objectives

Keep the code understandable, separate frontend from backend responsibilities, validate user input, protect credentials with password hashing, and document setup and deployment.

Success criteria

The application starts using the documented commands, the homepage loads, calculator input produces a result, API endpoints respond, and account/saved-estimate workflows work in a correctly configured environment.

6. Scope and Users

Intended users

Students learning web development, people exploring an illustrative loan scenario, and project evaluators inspecting a frontend-plus-backend implementation.

Included scope

The prepared project includes a web UI, styling, client-side JavaScript, an Express server, SQLite setup, authentication-related handling, saved-estimate operations, and setup documentation.

Out of scope

Actual loan underwriting, credit scoring, bank account access, lender comparison feeds, identity verification, payment processing, production-grade financial advice, and live AI integration.

Usage caution

Use sample data for demonstrations. Before public production use, the application needs privacy review, security testing, accessibility checks, legal review, monitoring, backups, and suitable deployment configuration.

7. Requirements Analysis

Functional requirements

Visitors can load the landing page and use the calculator. The frontend can send requests to backend endpoints. Account functions allow registration and login. Authenticated users can save estimates and delete saved items where supported.

Non-functional requirements

The application should be understandable, maintainable, responsive, and provide meaningful error messages. Sensitive configuration should be supplied through environment variables rather than committed to source control.

Software requirements

A modern browser, Node.js 20 or compatible version, npm, and a local environment able to write the SQLite database file are sufficient for development.

Constraints

SQLite is suitable for local demonstrations and modest workloads. Larger multi-instance deployments may need a managed database. A host must support a Node process and persistent storage if SQLite data must survive redeployments.

8. System Architecture

Overview

The system follows a client-server architecture. The browser renders HTML and CSS and runs JavaScript. JavaScript sends HTTP requests to the Express API. The server validates and processes requests and reads or writes SQLite data.

Request lifecycle

User interacts with the page → frontend validates or packages inputs → API request is sent → Express matches a route → server-side checks run → SQLite is accessed if needed → JSON response returns → page updates.

Separation of concerns

The public directory contains user-facing assets. server.js contains backend routing and application logic. package.json declares dependencies and commands. .env.example documents configuration without embedding live secrets.

Deployment note

A static-only host such as GitHub Pages cannot run the Express backend. Full-stack deployment requires a Node-capable service and a database strategy. GitHub stores source code but does not automatically host the API.

9. Frontend Implementation

HTML structure

public/index.html defines the page structure and interactive elements. Semantic headings, forms, buttons, and labelled inputs help users understand each section and improve maintainability.

CSS styling

public/style.css controls layout, spacing, typography, colors, and responsive presentation. A separate stylesheet makes visual changes easier without mixing them with application logic.

Client-side JavaScript

public/app.js handles interactions, collects values, sends API requests, and updates the interface with results or messages. Client-side checks improve usability but cannot replace server-side validation.

Implementation principle

Frontend code must not store private secrets. Requests that change persistent data must be checked by the backend. All important permission checks belong on the server.

10. Backend Implementation

Node.js and Express

server.js creates the HTTP server and defines routes. Express organizes request handlers, parses JSON request bodies, serves static files, and returns JSON responses.

Middleware

The prepared project uses security-oriented middleware such as Helmet and rate limiting. Middleware runs before or around route handlers and can set response headers or restrict excessive requests.

Validation and errors

The server should check required fields, data types, value ranges, and authentication status before an operation. Errors should return useful status codes without exposing stack traces or secrets.

Database interaction

The server uses SQLite for local persistence. Database operations should use parameterized queries and avoid concatenating untrusted input into SQL.

11. Database Design and Data Handling

SQLite role

SQLite stores application records in a database file, avoiding a separate database server for local development. The application creates its database structure during setup or startup.

Main data concepts

The project’s main data concepts are user accounts and saved loan estimates. A user record associates credentials with an account identifier; an estimate record stores scenario inputs and related calculated values or metadata.

Relationships

A user may have multiple saved estimates. Each estimate should be associated with its owner so one account cannot view or delete another user’s records.

Data lifecycle

Inputs are received by the API, validated, and stored when requested. Users should be told what data is stored and how to remove it. Public deployment requires retention and deletion rules.

12. Loan Calculator Logic

Inputs

A typical loan estimate uses principal P, annual interest rate r, and tenure. If the annual rate is a percentage, the monthly rate is i = r / 12 / 100. Tenure in years becomes n = years × 12 monthly payments.

Monthly payment formula

For a standard amortizing loan with positive monthly interest, EMI = P × i × (1+i)^n / ((1+i)^n – 1). If the interest rate is zero, EMI = P / n. The calculator must handle invalid or zero tenure safely.

Interpretation

Estimated total repayment is EMI × n. Estimated interest is total repayment – P. Fees, insurance, variable rates, rounding rules, and lender-specific conditions may not be included.

Validation

Reject negative principal, invalid rate, zero or negative tenure, missing fields, and non-numeric input. The exact formula and output should be verified against the actual implementation before claiming a production financial calculator.

13. Authentication and Security

Password handling

The prepared project uses bcrypt-style password hashing rather than storing readable passwords. Password hashes should not be exposed in API responses or logs.

Token-based access

JWT authentication can identify a logged-in user on protected routes. The server must verify token signatures and expiry, and must not trust a user ID supplied only by the browser.

Environment variables

JWT_SECRET and private settings belong in a local .env file or the deployment provider’s secret manager. The .env file must not be committed to GitHub. .env.example should contain placeholders only.

Additional safeguards

Rate limiting and secure response headers reduce some risks but do not automatically make an application production-safe. HTTPS, validation, dependency updates, secure token storage, authorization checks, and testing are also important.

14. API Design and Main Workflows

API purpose

The API is the boundary between the browser and server-side functionality. It should use predictable paths, HTTP methods, status codes, and JSON response shapes.

Route groups

The prepared project is documented around health/status, authentication, and saved-estimate operations. Exact route names and payload fields should be taken from server.js and README.md in the project ZIP.

Estimate workflow

The user enters scenario values, the interface displays a result, and the user can save the scenario when signed in. The server validates the data and checks ownership before saving.

Account workflow

A new user submits registration details; the server validates them and stores a password hash. At login, the server checks the password and issues a token. Protected operations require a valid token and enforce record ownership.

15. User Interface and User Experience

Clarity

Financial interfaces should explain principal, annual interest rate, tenure, monthly payment, total repayment, and estimated interest. Users should not need backend knowledge to use the calculator.

Feedback

The interface should confirm successful operations and explain recoverable problems such as missing inputs or an expired session. Messages should not expose internal server details.

Responsive layout

CSS should adapt to different screen widths so the calculator remains usable on laptops, tablets, and mobile devices. Touch targets, readable contrast, and sensible form order improve usability.

Accessibility

Use explicit labels, keyboard-accessible controls, visible focus states, meaningful headings, and text that does not rely on color alone.

16. Testing and Quality Assurance

Manual tests

Load the home page; enter valid inputs; test zero or invalid values; try registration and login; save and delete an estimate; and verify signed-out users cannot access protected data.

API tests

Test successful responses, missing fields, malformed JSON, invalid credentials, missing or expired tokens, and attempts to access another user's record. Check status codes and error messages.

Security tests

Check for leaked secrets, SQL injection risks, weak authorization, excessive request rates, unsafe error output, and vulnerable dependencies. Use sample data rather than real identity or financial records.

Acceptance checklist

The server starts; the page loads; calculator outputs are sensible; data persists in the intended environment; authentication and ownership checks work; and setup instructions work on a clean machine.

17. Installation, Execution, and GitHub

Prerequisites

Install Node.js 20 or a compatible supported release and npm. Extract the ZIP into a project folder and open a terminal in that folder.

Local setup

Run npm install. Copy .env.example to .env. Set a long random JWT_SECRET in .env. Run npm start, then open http://localhost:3000. Follow README.md if the project scripts differ.

GitHub workflow

Create a repository, add the source files, commit them, and push to GitHub. Exclude node_modules, the real .env file, local databases containing personal data, and secrets using .gitignore.

Hosting

Deploy the Node server to a platform that supports Node.js. Configure environment variables and persistent database storage. GitHub Pages can host static assets only; it cannot run this Express API.

18. Limitations, Risks, and Responsible Use

Financial limitations

A calculator result is not a loan offer, credit decision, eligibility determination, or guarantee. Lenders can use different fees, interest methods, eligibility rules, and approval processes.

Technical limitations

SQLite and a single Node process are appropriate for a demo, but public production use may require stronger database operations, backups, monitoring, structured logs, automated tests, and deployment hardening.

Privacy risks

Account information and saved scenarios can be sensitive. Collect only necessary data, explain retention, protect data in transit, restrict access, and provide deletion procedures.

Integration boundary

The prepared project does not claim to connect to banks, credit bureaus, payment networks, or live AI services. Such integrations require authorization, secure credentials, compliance review, and separate testing.

19. Future Enhancements and Recommendations

Product enhancements

Add scenario comparison, downloadable amortization schedules, clear fee assumptions, and a financial glossary. Allow users to compare rates and tenures without presenting a result as guaranteed.

Engineering enhancements

Add automated unit and API tests, input schemas, database migrations, structured logging, backup and restore procedures, and continuous integration. Consider a managed relational database for multi-instance production hosting.

Trust and transparency

Publish a plain-language privacy notice, calculator methodology page, disclaimer, and contact/support details. Clearly distinguish user-entered numbers from system estimates.

Potential integrations

A future version might integrate verified lender data or an external financial service after authorization and due diligence. Any AI assistant should be clearly labelled and prevented from inventing rates or approvals.

20. Conclusion and References

Conclusion

CredNexa demonstrates how a financial-planning interface can be built as a full-stack web project. Its value lies in making an illustrative loan scenario easier to explore while teaching frontend development, API design, database persistence, authentication, and deployment.

Key takeaway

The project is a starting point for learning and demonstration. It should be tested and reviewed before public use, and loan outputs should remain clearly identified as estimates.

Project materials

The primary implementation reference is the prepared CredNexa ZIP package: server.js, public/index.html, public/style.css, public/app.js, package.json, .env.example, .gitignore, and README.md. Consult these files for exact route names, code, and setup commands.

Technical references

Node.js documentation: https://nodejs.org/docs/ ; Express documentation: https://expressjs.com/ ; SQLite documentation: https://www.sqlite.org/docs.html ; OWASP guidance: https://owasp.org/ . These are general technical references, not evidence of certification or regulatory approval.

Final note

This report documents the recreated project package and its intended design. Verify the actual code and deployment environment before making claims about live integrations, compliance, or production behavior.